

FastAPI

August 3, 2026

1 FastAPI

1.1 Some notes first

- you should really stop using pip and start using uv (uv is a pip alternative, please go and watch: <https://youtu.be/AMdG7IjgSPM?si=2NCbkP6E1kPtUVfB>)
- Once you have uv installed and working, you can hit the following commands easily:
 - uv init (init command initializes a python/uv project in the folder)
 - uv add 'fastapi[standard]' (this is the same as pip install fastapi)

1.2 Some basics before starting

- The goal we are trying to achieve here is simple. We have to setup a url/endpoint that the frontend (javascript or typescript) can hit with fetch or axios to make requests, like a GET or POST or anything like that
- Here is some simple code for the frontend, this is kinda like that code that the frontend engineer will be writing:

```
[ ]: async function getData(): Promise<void> {  
  try {  
    const response = await fetch("http://127.0.0.1:8000/getdata")  
    if (!response.ok) {  
      return  
      // throw some error  
    }  
    const data = await response.json()  
    console.log(data)  
    // now here we will do something with the data, probably use a setState_  
    ↪ function to set it in a state  
  } catch (e) {  
    console.log("yo we got some error ma dawg: ${e}")  
  }  
}
```

- now, where does that `http://localhost:8000/getdata` come from? Well, we basically create that endpoint using fastapi

1.3 Setup app and simple endpoint

```
[1]: # step 1: create a fastapi application with the FastAPI constructor
from fastapi import FastAPI

app = FastAPI()

[2]: # step 2: we setup an endpoint using decorators, you don't need to learn what
      ↪ decorators are or how they work, just use this code simply:

      # this creates the default route, if you visit localhost:8000 now, you will
      ↪ have the following dictionary returning
@app.get("/")
def get_data():
    return {"hello there": "yo yo mate, whats popping. this is the default
    ↪ route"}

@app.get("/getdata")
def getdata():
    return {"id": 12705, "childName": "Junaid", "childHeartRate": 467}
```

- lets try to breakdown step 2 further:
 - 1. we use @app.get or @app.post or whatever decorator to make a new endpoint
 - 2. we then define a function under the decorator that does something (returns some data, receive some data, etc etc)
 - 3. Its convention to return a dictionary from these endpoint functions because python dictionaries map nicely to JSON and also typescript types on the frontend. Later we will learn about pydantic data models which map even more perfectly to the frontend typescript types
- lets see the output

Note:

- you run fastapi code in .py files, not .ipynb files. So make sure to make a .py file and write the code in that.
- you can hit `uv run fastapi dev` to run the code.
- once running, you will see something like this in the terminal:

4) Local dev vs prod

- **Dev:** allow **localhost (line 3000)** and **127.0.0.1 (line 3000)**
- **Prod:** allow only your real frontend domain

5) What CORS is NOT

- It is **not** authentication.
- It only affects **browsers**.
- Your ESP32 or curl requests are **not blocked** by CORS.

- visit <http://127.0.0.1:8000> and you will see:

The screenshot shows a REST client interface for a POST request to `/readings`. The interface includes a header bar with the method `POST` and the endpoint `/readings`, along with a 'Create Reading' link. Below the header, there is a 'Parameters' section with a 'Cancel' button and a note 'No parameters'. The 'Request body' section is marked as 'required' and has a dropdown menu set to `application/json`. The request body is a JSON object:

```
{  "device_id": "string",  "temperature_c": 0,  "humidity": 0,  "status": "ok"}
```

. At the bottom of the request section is a large blue 'Execute' button. Below the request section is a 'Responses' section.

- we can see the dictionary we returned.
- visit the seconddd endpoint we made at <http://127.0.0.1:8000/getdata> and we will see:

Request body required

Edit Value | Schema

```
{
  "device_id": "string",
  "temperature_c": 0,
  "humidity": 0,
  "status": "ok"
}
```

- That is the core and simplest concept, we make endpoints, we send or receive data from those endpoints or do many other things as well.
- now lets cover someother basics:
 - how to receive data on an endpoint
 - how to make parameterized endpoints
 - how to view documentation

1.4 Viewing and testing our endpoint

- you can visit <http://127.0.0.1:8000/docs> and you will see:

4) Local dev vs prod

- Dev: allow **localhost (line 3000)** and **127.0.0.1 (line 3000)**
- Prod: allow only your real frontend domain

5) What CORS is NOT

- It is **not** authentication.
- It only affects **browsers**.
- Your ESP32 or curl requests are **not blocked** by CORS.

- This is an auto generated documentation page where one can view and test out the api endpoint we just created. The frontend engineer should first use this and test it out and see if everything works well, then they should write the frontend code to hit the endpoints.

The screenshot shows an API testing tool interface. At the top, it indicates a **POST** request to `/readings` with the description 'Create Reading'. Below this, there is a 'Parameters' section which is currently empty, showing 'No parameters'. A 'Request body' section is marked as 'required' and has a dropdown menu set to 'application/json'. The request body field contains a JSON object:

```
{  "device_id": "string",  "temperature_c": 0,  "humidity": 0,  "status": "ok"}
```

. At the bottom of the interface, there is a large blue button labeled 'Execute'. Below the button, a 'Responses' section is visible but currently empty.

- we can press the try it out button to test the endpoint and see output and manipulate parameters etc

1.5 create a parameterized route

- lets say the frontend wants to view the data of a specific child

```
[3]: # some dummy data
db = {
  12701: {"weight": 3.5, "content": "furniture", "status": "in transit"},
  12702: {"weight": 1.2, "content": "books", "status": "delivered"},
  12703: {"weight": 7.8, "content": "electronics", "status": "pending"},
  12704: {"weight": 2.0, "content": "clothes", "status": "in transit"},
  12705: {"weight": 5.5, "content": "appliances", "status": "shipped"},
}
```

```
12706: {"weight": 0.9, "content": "toys", "status": "delivered"},
}
```

```
[ ]: from fastapi import HTTPException, status

@app.get("/getdata/{id}")
def getdata_by_id(id: int):
    # some code to fetch that exact child by id from the database. but since
    ↪our "db" is just the dict above, we can do this:
    if id not in db:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Given id does not
            ↪exist"
        )
    return db[id]
```

- Lets review:
 - 1. we use {} in the decorator to create a dynamic endpoint
 - 2. we receive that dynamic part of the endpoint into our function by having parameter in the function with the same name.
 - 3. the : int is type annotation that enforces the type of data
 - 4. we return a proper http error if the id does not exist
 - 5. we return the data if everything goes fine
- we can now visit <http://127.0.0.1:8000/docs> again after saving the code and we will see:

```
4) Local dev vs prod

• Dev: allow localhost (line 3000) and 127.0.0.1 (line 3000)

• Prod: allow only your real frontend domain

5) What CORS is NOT

• It is not authentication.

• It only affects browsers.

• Your ESP32 or curl requests are not blocked by CORS.
```

- there is a third new get endpoint at `/getdata/{id}`. we can of course test it out in the documentation with the try it out button. or we could just visit it directly:

4) Local dev vs prod

- Dev: allow **localhost (line 3000)** and **127.0.0.1 (line 3000)**
- Prod: allow only your real frontend domain

5) What CORS is NOT

- It is **not** authentication.
- It only affects **browsers**.
- Your ESP32 or curl requests are **not blocked** by CORS.

- if we visit an id that does not exist. it returns a proper error:

POST /readings Create Reading

Parameters

No parameters

Request body *required* application/json

Edit Value | Schema

```
{
  "device_id": "string",
  "temperature_c": 0,
  "humidity": 0,
  "status": "ok"
}
```

Execute

Responses

1.6 Query params

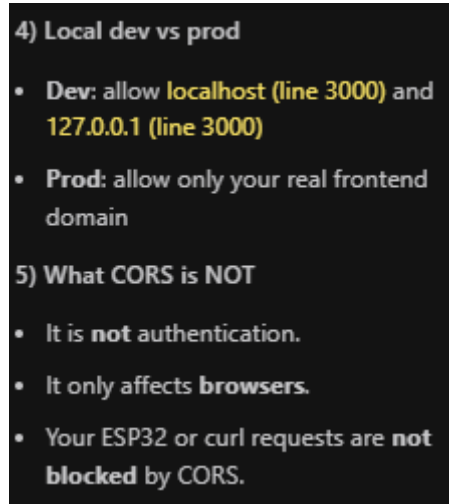
- The frontend primarily gives us data in 2 ways:
 - route params: `http://127.0.0.1:8000/getdata/{id}` (what we did above with id)
 - query params: what we are going to do now

```
[ ]: # no dynamic endpoint needed for query params
```

```
@app.get("/getdata/anything")
def getdata_with_query_params(sort_by: str):
    if sort_by not in ("asc", "dsc"):
        raise HTTPException(
            status_code=400, detail="invalid sort order option selected"
        )
```

```
return sort_by
```

- Lets break this down:
 - 1. note how the url in the decorator is not special, it is just a simple static route
 - 2. but the function does have a parameter and therefore we can pass it arguments
 - 3. if you visit getdata/anything and do not give any query params, you will get this:



- 4. lets hit it again but this time. lets give it a query param:

POST /readings Create Reading

Parameters Cancel

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "device_id": "string",
  "temperature_c": 0,
  "humidity": 0,
  "status": "ok"
}
```

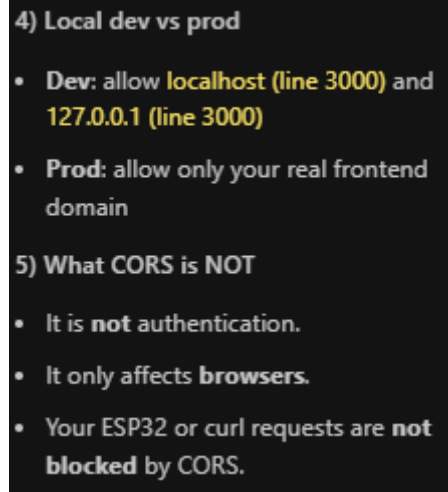
Execute

Responses

- our function was able to grab the query param from the url and then we used it in the function
- **note:** declare static route functions above dynamic route ones. if you have the following two functions:
 - getdata/anything
 - getdata/{id}
- the second one is dynamic, if you declare it above the getdata/anything function in the .py file, your getdata/anything will also get intercepted by getdata/{id} since {id} is dynamic and can accept anything.

1.7 Pydantic Data Modal

- we saw previously that we set the type of data we wanna receive using type annotation (eg id: int) and then if we give some other data type then it gives an error:



- but the frontend might need to send us data of an entire form. for example we have a form on our frontend and it has several fields like device_id, temperature, humidity and status etc. What do we do in that case? We will use pydantic data modals. These are like the models from flutter or types from typescript

```
[ ]: from pydantic import BaseModel, Field
from typing import Literal

class SensorReading(BaseModel):
    device_id: str = Field(min_length=3)
    temperature_c: float
    humidity: float
    status: Literal["ok", "warn", "error"]

@app.post("/readings")
def create_reading(payload: SensorReading):
    # the data validation will be done automatically now because of the
    ↪ pydantic modal but we can do further validation with if cases and
    ↪ HTTPException:
    if payload.temperature_c < -50 or payload.temperature_c > 150:
        raise HTTPException(status_code=400, detail="temperature out of range")
    return {"stored": True, "data": payload}
```

- as you can see, a pydantic modal is just a simple python class that inherits the BaseModel class from pydantic library and then we simply provide the fields we have in that class.
- we get other utilities from pydantic too like the Field method that can be used to specify the minimum length etc and put validations on the fields easily.

- after defining the pydantic modal, we assign that class to our parameter in the function so now our function accepts that entire pydantic data modal from the frontend. The frontend will send this to us kinda like this:

```
[ ]: // a typescript type
type SensorReading = {
  device_id: string;
  temperature_c: number;
  humidity: number;
  status: "ok" | "warn" | "error";
};

async function sendReading() {
  const payload: SensorReading = {
    device_id: "esp32-01",
    temperature_c: 24.7,
    humidity: 52.1,
    status: "ok",
  };

  const res = await fetch("http://127.0.0.1:8000/readings", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload), // this is the data that we need to receive
    ↪in the fastapi endpoint function and then map this onto our pydantic data
    ↪modal
  });

  if (!res.ok) {
    const err = await res.text();
    throw new Error(err);
  }

  const data = await res.json();
  console.log(data);
}

sendReading().catch(console.error);
```

But how do we test?

- if we had simple query params or even a parameterized route then we could of course write something like `http://localhost:8000/getdata/123` or `http://localhost:8000/getdata/anything?sort_by=asc` etc etc in the url and test out our endpoint by giving it data through the route param or query param. But this time, we want to send a full form/typescript type to our fastapi endpoint. For this, we will need to go back to our `http://localhost:8000/docs` endpoint, and there we can easily write a json body

4) Local dev vs prod

- **Dev:** allow **localhost (line 3000)** and **127.0.0.1 (line 3000)**
- **Prod:** allow only your real frontend domain

5) What CORS is NOT

- It is **not** authentication.
- It only affects **browsers**.
- Your ESP32 or curl requests are **not blocked** by CORS.

- we can see the new endpoint and we can also see that it is a post endpoint, not get.

The screenshot shows a REST client interface for a POST request to the endpoint `/readings`. The interface includes a header bar with the method `POST` and the endpoint `/readings`, along with a 'Create Reading' link. Below the header, there is a 'Parameters' section with a 'Cancel' button and a note 'No parameters'. The 'Request body' section is marked as 'required' and has a dropdown menu set to `application/json`. The request body is displayed in a text area with a JSON schema:

```
{  "device_id": "string",  "temperature_c": 0,  "humidity": 0,  "status": "ok"}
```

. At the bottom of the request section is a large blue 'Execute' button. Below the request section is a 'Responses' section.

- upon pressing try it out, we can see and edit the request body and then press execute and see the response the endpoint gives. I will share a screenshot of the request body data and the response below:

Request body required

Edit Value | Schema

```
{
  "device_id": "string",
  "temperature_c": 0,
  "humidity": 0,
  "status": "ok"
}
```

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/readings' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "device_id": "string",
    "temperature_c": 0,
    "humidity": 0,
    "status": "ok"
  }'
```

Request URL

http://localhost:8000/readings

Server response

Code Details

200

Response body

```
{
  "stored": true,
  "data": {
    "device_id": "string",
    "temperature_c": 0,
    "humidity": 0,
    "status": "ok"
  }
}
```

Response headers

```
content-length: 94
content-type: application/json
date: Mon, 02 Feb 2020 06:54:39 GMT
server: uvicorn
```

- you can see that we got a nice response, show both keys that we returned from our fastapi endpoint function `create_readings`. The frontend engineer can test the api and see the response body the fastapi endpoint will be giving from here and then destructure the response from the `fetch/axios` call accordingly
- lets now break a few things and see how the errors are shown:

Edit Value | Schema

```
{
  "device_id": 1232,
  "temperature_c": 0,
  "humidity": 0,
  "status": "ok"
}
```

- I gave an int as the device_id even though the pydantic modal expects a string:

Code	Details
422	Error: Unprocessable Content Response body <pre>{ "detail": [{ "type": "string_type", "loc": ["body", "device_id"], "msg": "Input should be a valid string", "input": 1232 }]}</pre> Response headers <pre>content-length: 114 content-type: application/json date: Mon, 02 Feb 2026 06:58:24 GMT server: uvicorn</pre>

- as you can see, we get a nice error message saying input should be a valid string and an error code too. the frontend developer can access these error codes and messages and make try catch or if-else blocks using these of course.

Edit Value | Schema

```
{  
}
```

Execute

Code	Details
422	Error: Unprocessable Content Response body <pre>{ "detail": [{ "type": "missing", "loc": ["body", "device_id"], "msg": "Field required", "input": {} }, { "type": "missing", "loc": ["body", "temperature_c"], "msg": "Field required", "input": {} }, { "type": "missing", "loc": ["body", "humidity"], "msg": "Field required", "input": {} }] }</pre>

- I tried to hit the endpoint with an empty json body and the pydantic data modal automatically returns that the fields were missing.
- So, in summary, try to always use pydantic data modals and accept and send data from your endpoints using them as they automate and standardize several things like we saw above.
- I will now paste in some gpted code of CRUD endpoints created using fastapi:

```
[ ]: from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field
from typing import Literal

app = FastAPI()

class SensorReading(BaseModel):
```



```

device_id: str = Field(min_length=3)
temperature_c: float
humidity: float
status: Literal["ok", "warn", "error"]

# in-memory store (replace with real DB later)
readings_db: dict[int, SensorReading] = {}
next_id = 1

# put/update endpoint
@app.put("/readings/{reading_id}")
def update_reading(reading_id: int, payload: SensorReading):
    if reading_id not in readings_db:
        raise HTTPException(status_code=404, detail="reading not found")
    readings_db[reading_id] = payload
    return {"id": reading_id, "data": payload}

# delete endpoint
@app.delete("/readings/{reading_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_reading(reading_id: int):
    if reading_id not in readings_db:
        raise HTTPException(status_code=404, detail="reading not found")
    del readings_db[reading_id]
    return None

```

1.8 Connecting a database with our backend layer

- We will obviously need to connect a database with our frontend+backend application. Lets say we use something simple like supabase. The setup would be very simple:
 - 1. we hit the command: `uv add supabase`
 - 2. we create our `.env` file and store our supabase url and supabase service key there which we get from our supabase project settings from the supabase website
 - 3. lastly we just create the supabase client and start working
 - this is what the code would look like somewhat:

```

[ ]: import os
from supabase import create_client, Client

SUPABASE_URL = os.getenv("SUPABASE_URL")
SUPABASE_SERVICE_KEY = os.getenv("SUPABASE_SERVICE_KEY")

supabase: Client = create_client(SUPABASE_URL, SUPABASE_SERVICE_KEY)

```

```

# Insert a row
def insert_reading(data: dict):
    return supabase.table("readings").insert(data).execute()

# Select rows
def list_readings():
    return (
        supabase.table("readings").select("*").order("created_at", desc=True).
        ↪execute()
    )

```

- and after that we just make fastapi endpoint that wrap these supabase helper functions

```

[ ]: from fastapi import FastAPI, HTTPException
    from pydantic import BaseModel

app = FastAPI()

class ReadingIn(BaseModel):
    device_id: str
    temperature_c: float
    humidity: float
    status: str

@app.post("/readings")
def create_reading(payload: ReadingIn):
    res = supabase.table("readings").insert(payload.model_dump()).execute()
    if not res.data:
        raise HTTPException(status_code=500, detail="Insert failed")
    return res.data[0]

@app.get("/readings")
def get_readings():
    res = supabase.table("readings").select("*").execute()
    return res.data

```

- a common confusion some people have at this point would be: “Why am I using FastAPI bro? I can just directly use the supabase library in typescript and I have my whole backend there”
- The answer can be given in several diff ways:
 - Secret keys: Any logic requiring a Supabase service key must stay on the server. Frontend code can only use anon keys.
 - IoT / ESP32: Devices should hit your backend, not your database directly. You validate, authenticate, and normalize payloads there.

- Business logic: You often need custom rules, validation, and side-effects (alerts, transforms, analytics) before storing data.
- Rate limiting & security: Your API can block abuse, log requests, and apply stricter auth.
- Data shaping: Backend can return exactly what frontend needs (joins, aggregations, caching).
- Future flexibility: If you swap DBs or add services, frontend doesn't change.

1.9 CORs

- so far, we have been working with fastapi on our local computer, we go to the localhost:8000 port and everything works fine. But the thing is if we were not on our local computer working like this, fastapi would block anyone from fetching/sending or doing anything with the endpoints.
- We essentially need to tell fastapi “hey, the frontend is hosted at localhost:3000, if that url hits you with a request then you accept it ok? if its someother url, deny it”.
- for this functionality (i.e: allowing our frontend to hit our backend) we will use cors. Lets first see the code:

```
[ ]: from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

# Allow your frontend URL(s)
origins = [
    "http://localhost:3000",
    "http://127.0.0.1:3000",
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins, # who can access
    allow_credentials=True, # allow cookies/auth headers
    allow_methods=["*"], # GET/POST/PUT/DELETE/etc
    allow_headers=["*"], # headers like Authorization, Content-Type
)
```

- what we have done is introduce middleware into our fastapi backend, this middleware allows some origins (urls basically) to communicate with our fastapi endpoints, and rejects all other origins.
- next and react locally run on port 3000, if you are using vite then its on port 5173 I think (later, when we host our frontend on the web with a real domain, we will put that here). that is why i wrote the localhost:3000 url above.
- since we intend to work with esp, the following points are important:

4) Local dev vs prod

- Dev: allow **localhost (line 3000)** and **127.0.0.1 (line 3000)**
- Prod: allow only your real frontend domain

5) What CORS is NOT

- It is **not** authentication.
- It only affects **browsers**.
- Your ESP32 or curl requests are **not blocked** by CORS.

1.10 Context Managers

```
[ ]: class Database:
    def __enter__(self):
        print("Code that will execute after the with keyword")
        return self

    def __exit__(self, *arg):
        print("Code that will execute when you dedent out of the context_
↳manager")
```

- we can also have function based context managers

```
[ ]: from contextlib import contextmanager

@contextmanager
def managed_db():
    db = Database()
    try:
        yield db
    finally:
        print("done")

with managed_db() as db:
    pass
    # something something
```

1.11 Annotated Types

- pass in metadata with a dtype

```
[ ]: from typing import Annotated

temp: Annotated[float, "Celsius"] = 37.4
```

1.12 SQLAlchemy

```
[ ]: from datetime import datetime
from enum import Enum
from sqlmodel import SQLModel, Field

class ShipmentStatus(str, Enum):
    placed = "placed"
    in_transit = "in_transit"
    out_for_delivery = "out_for_delivery"
    delivered = "delivered"

class Shipment(SQLModel, table=True):
    # __tablename__ = "shipment"
    id: int | None = Field(default=None, primary_key=True)
    content: str
    weight: float = Field(le=25, gt=1)
    destination: int
    status: ShipmentStatus = Field(default=ShipmentStatus.placed)
    estimated_delivery: datetime
```

1.12.1 Session based CRUD and creating connection

```
[ ]: from typing import Annotated

from fastapi import Depends
from sqlalchemy import create_engine
from sqlmodel import Session, SQLModel

engine = create_engine(
    url="sqlite:///shipmentdb.db", echo=True, connect_args={"check_same_thread":
    ↪ False}
)

def create_db_tables():
    SQLModel.metadata.create_all(bind=engine)

def get_session():
    with Session(bind=engine) as session:
```

```
yield session
```

```
SessionDep = Annotated[Session, Depends(get_session)]
```

1.13 Dependency Injection

- providing data/stuff your endpoint might need

1.13.1 Example

- Endpoint needs a DB session.
- Instead of creating session manually in every route, you declare it as a dependency.
- FastAPI calls the dependency function, passes result into your endpoint, and handles cleanup.

```
[ ]: def get_session():  
    with Session(engine) as session:  
        yield session  
  
SessionDep = Annotated[Session, Depends(get_session)]  
  
@app.get("/shipment/{id}")  
def get_shipment(id: int, session: SessionDep): ...
```

1.14 Lifespan and Generators

- need to run some code when the fastapi application starts or stops? Just pass in a lifespan method.
- the concept is the same as context managers. consider the example below
- we make sure our tables are created before we write yield so before the server can accept requests, the tables are created.
- we can do any sorta clean up after yield when the server is stopping
- lifespan is simply a context manager around the entire fastapi app
- Why FastAPI uses generators: setup before yield, teardown after yield.

```
[ ]: from contextlib import asynccontextmanager  
from rich import panel, print  
  
@asynccontextmanager  
async def lifespan_handler(app: FastAPI):  
    print(panel.Panel("Server is started", border_style="green"))  
    create_db_tables()  
    yield  
    print(panel.Panel("Server stopped", border_style="red"))
```

```
app = FastAPI(lifespan=lifespan_handler)
```

1.15 Engine vs Session vs SessionMaker

- **engine** = database infrastructure (pool + dialect + connection management)
- **session** = one request's database workspace (query/add/commit)
- **sessionmaker** = factory to create sessions with same config every time

```
[ ]: from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession, \
      ↪ async_sessionmaker

engine = create_async_engine(DATABASE_URL, echo=True)

AsyncSessionLocal = async_sessionmaker(
    bind=engine,
    class_=AsyncSession,
    expire_on_commit=False,
)
```

1.16 Async

- async functions in python are also called coroutines
- to get the output from a coroutine, you have 2 ways:
 - either the async code is wrapped inside an async function and is awaited
 - or you use `asyncio.run` to run it
- you can have a coroutine and then do `await` in it, that would run the lines inside the function. but when calling the entire function itself (which itself is a coroutine) you will have to use `asyncio.run`

```
[ ]: import asyncio

async def server():
    # await something()
    pass

asyncio.run(server())
```

- use `async.create_task()` with `asyncio.wait(list)`

```
[ ]: async with asyncio.TaskGroup() as task_group:
      task_group.create_task()
```

```
[ ]: import asyncio
```

```

async def fetch(name, delay):
    await asyncio.sleep(delay)
    return name

async def main():
    async with asyncio.TaskGroup() as tg:
        t1 = tg.create_task(fetch("shipment-1", 1))
        t2 = tg.create_task(fetch("shipment-2", 1))
    print(t1.result(), t2.result())

```

```

[ ]: # 1) Async helps across requests (no create_task needed)
# FastAPI can serve other requests while this one awaits I/O.
@app.get("/io")
async def io_bound():
    data = await external_api_call() # yields control while waiting
    return {"data": data}

# 2) Sequential awaits inside ONE request (still async, but not parallel)
@app.get("/sequential")
async def sequential():
    a = await fetch_user() # wait
    b = await fetch_orders() # then wait
    return {"user": a, "orders": b}

# 3) Parallel work inside ONE request with create_task + gather

@app.get("/parallel")
async def parallel():
    t1 = asyncio.create_task(fetch_user())
    t2 = asyncio.create_task(fetch_orders())
    user, orders = await asyncio.gather(t1, t2)
    return {"user": user, "orders": orders}

# 4) TaskGroup version (safer structured concurrency)

@app.get("/parallel-taskgroup")
async def parallel_taskgroup():
    async with asyncio.TaskGroup() as tg:
        t1 = tg.create_task(fetch_user())

```



```
t2 = tg.create_task(fetch_orders())
return {"user": t1.result(), "orders": t2.result()}
```

1.17 Generator Dependency in FastAPI

- We use `yield` (not `return`) so FastAPI can run cleanup after request ends.
- Code before `yield` = setup
- Code after `yield` = teardown

1.18 More on Dependency Injection

- lets recap dependency injection properly and add a few things we didnt cover yet
- basically, an object receives another object that it needs (this other object is called a dependency) instead of creating it itself. thats the entire idea behind DI
- you use `Depends(your_function)` as an argument in your api function, fastapi automatically runs that dependency function first and then injects its return value straight into your endpoint parameter. this is great for reusing db sessions, handling authentication, and making route testing way simpler
- common convention is using the `Annotated` type like this: `db: Annotated[Session, Depends(get_db)]`. so fastapi runs `get_db` by itself, and the return type of `get_db` (which is `Session`) is what gets stored in the `db` variable. all done automatically by fastapi, we never call `get_db` ourselves anywhere
- `Depends` is basically a runtime marker that tells fastapi “go run the function inside me first, then inject its return value into my endpoint”
- if multiple routes use the exact same dependency, fastapi automatically caches the result so it doesnt run the dependency again and again for the same request. if you dont want that caching behaviour, just do `Depends(get_db, use_cache=False)`
- if every single route in a router needs the same dependency over and over, dont keep repeating `Depends()` in each route. just do this on the router itself: `APIRouter(dependencies=[Depends(xyz)])`. these still run for every route in that router, but their return value wont get injected as an argument into your route functions. this is mostly used for access control and security, something like:

```
[ ]: from fastapi import APIRouter, Depends

admin_router = APIRouter(dependencies=[Depends(verify_admin_token)])
```

- at this point i got kinda confused, because dependency injection sounded exactly like middleware to me (like from django or any other backend framework). but heres the actual difference:
 - middleware in fastapi runs for ALL your endpoints by default
 - dependency injections can be configured to only run for certain endpoints/routers, way more flexible than middleware
 - if you want to pass data from your middleware into your endpoint function, you gotta do a workaround, like stuffing the data into `request.state` and then manually pulling it back out inside your endpoint
 - dependency injections dont need any of that workaround since they run directly inside the python/fastapi ecosystem, so they can just pass their return value straight into your

endpoint as a normal function argument

- side note: we never had to write any of this session setup/teardown stuff in django, because django handles db sessions completely automatically for us, the whole db session lifecycle is fully owned by django. fastapi doesnt do this for us, so we gotta manage it ourselves with dependency injection + generators, which is exactly what we did above with `get_session`

1.19 MODULE 9 Recap: Async Programming

- quick recap plus a couple new bits on async before we move onto postgres
- if you ever wanna time how long a piece of code takes, use `time.perf_counter()` from the time module. just grab the time before and after your code runs and subtract the two
- in python, an async function is called a coroutine (we already know this from above)
- calling an async function doesnt run it, it just gives you back a coroutine object. to actually get the return value out of a coroutine you have 2 options:
 - wrap it inside another async function and `await` it there
 - or use `asyncio.run()` to run it directly from outside any async function
- so basically, if youre already inside an async function you just `await` stuff. but if youre running the whole coroutine itself from the very top level (like the entrypoint of your script), youll need `asyncio.run()`
- heres an older way of firing off a bunch of tasks together, before task groups existed:

```
[ ]: import asyncio

requests = [asyncio.create_task(run_endpoint(endpoint)) for endpoint in endpoints]
done, pending = await asyncio.wait(requests)

for task in done:
    print(task.result())
```

- and like we already saw above, `TaskGroup` is the newer and safer way of doing the exact same thing, so just prefer `TaskGroup` going forward over this `asyncio.wait` pattern

1.20 MODULE 10 Recap: Postgres

- this whole module is about hooking up a real postgres database to our app instead of sqlite, and cleaning up our code with routers plus a service layer. lets go over the whole thing properly from the top
- first thing, we dont wanna hardcode our db credentials directly in our code. so we use `pydantic-settings` to read them from a `.env` file instead:

```
[ ]: from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    # write the field names exactly the same as in your .env file, and hint the type
    POSTGRES_PASSWORD: str
    POSTGRES_PORT: int
```

```

# now point your class to the env file
model_config = SettingsConfigDict(
    env_file="./.env", env_ignore_empty=True, extra="ignore"
)

settings = Settings()
print(settings.POSTGRES_PASSWORD)

```

- basically we make a class that inherits BaseSettings, write out the field names + types (they gotta match whats in the .env file exactly), then point model_config at our .env file. now settings.POSTGRES_PASSWORD just works, no os.getenv() nonsense needed anywhere
- we already covered engine, session and sessionmaker above, but as a quick reminder:
 - engine = the database infrastructure itself (connection pool + dialect + connection management)
 - session = one single requests workspace to talk to the db (query/add/commit)
 - sessionmaker = a factory that keeps making sessions with the same config every time, so we dont repeat ourselves
- and all of this gets used inside our get_session dependency, which we already wrote above
- next thing, once our endpoints start growing, we dont wanna keep dumping everything into main.py. so we make a router instead:

```

[ ]: from fastapi import APIRouter

router = APIRouter(prefix="/shipment", tags=["Shipment"])

@router.get("/{id}")
async def get_shipment(id: int, session: SessionDep):
    ...

```

- and then we just plug that router into our main app:

```

[ ]: app.include_router(router)

```

- next up, we dont want our endpoint functions doing all the db work themselves either, thats gonna get messy fast. so we make a service class that handles all of that, and our endpoint just calls into it. make a folder called services, chuck an **init.py** in there, then make a file like shipment.py:

```

[ ]: class ShipmentService:
    def __init__(self, session: AsyncSession):
        self.session = session

    async def get(self, id: int) -> Shipment:
        return await self.session.get(Shipment, id)

    async def add(self, shipment_create: ShipmentCreate) -> Shipment:

```

```

        # implementation here
        ...

    async def update(self, id: int, shipment_update: ShipmentUpdateModel) -> Shipment:
        # implementation here
        ...

    async def delete(self, id: int) -> None:
        # implementation here
        ...

```

- now we just instantiate this service inside our endpoint and use it. or even better, we inject it too using dependency injection, same as we did with the session:

```

[ ]: from typing import Annotated

from fastapi import Depends

SessionDep = Annotated[AsyncSession, Depends(get_session)]

def get_shipment_service(session: SessionDep):
    return ShipmentService(session)

ServiceDep = Annotated[ShipmentService, Depends(get_shipment_service)]

@router.get("/{id}")
async def get_shipment(id: int, service: ServiceDep):
    return await service.get(id)

```

- notice how ServiceDep itself depends on SessionDep, this is called dependency chaining. one dependency depending on another dependency, and fastapi just resolves the whole chain for us automatically, we dont gotta worry about ordering or anything, it just works
- and thats basically the whole flow: router -> endpoint -> service (injected) -> session (injected) -> db. nice clean separation, every layer only worries about its own job

1.21 Final Project Structure

- and if we put literally everything from this entire notebook together properly (routers, schemas, services, database, config), we end up with a project structure that looks like this:
 - main.py -> creates the app, sets up the lifespan handler, includes the router
 - api/router.py -> all our /shipment routes (get, post, patch, delete)
 - api/dependencies.py -> SessionDep, ServiceDep, the whole dependency chain
 - api/schemas/shipment.py -> our pydantic models (ShipmentCreate, ShipmentUpdateModel, ShipmentRead)

- database/models.py -> the SQLAlchemy Shipment table + the ShipmentStatus enum
- database/session.py -> engine, sessionmaker, get_session, create_db_tables
- config.py -> our pydantic-settings class that reads from .env
- services/shipment.py -> ShipmentService with all the actual CRUD logic
- this is exactly the shipment api we built while going through this whole course, so if anything in this notebook ever feels confusing, just go open up the actual files in fastapi_practice/ and match them up against the notes above, it'll click

1.22 A Few More Odds and Ends

- few small things i used along the way that i didnt properly note down anywhere, adding them here so nothing gets lost
- **PUT vs PATCH**, people mix these up a lot:
 - PUT means you remove the existing data and replace it entirely with whatever new data you give it. so you basically have to send every field again, even the ones that didnt change
 - PATCH means a true partial update, you only send the fields you actually wanna change, and the rest stays untouched
 - you can implement patch two ways: either accept a bunch of individual Optional params and only update the ones that arent None, or just accept a proper pydantic body model where the fields are optional. the second way is way cleaner
- **response_model and status_code on routes**: on our route decorators we can do stuff like `@router.post("/", response_model=ShipmentRead, status_code=201)`. `response_model` tells fastapi exactly what shape of data to send back (it filters out anything extra and validates it), and `status_code` just sets the default http status for that response (201 means "created", makes sense for a POST that creates something)
- **sqlmodel_update()**: when updating a row with sqlmodel, instead of manually going field by field and doing `shipment.status = update.status` etc, you can just do `shipment.sqlmodel_update(update_data.model_dump(exclude_none=True))`. this takes a dict and updates only the matching fields on the model in one go. `exclude_none=True` on `model_dump` makes sure we dont overwrite fields with None when the user didnt actually provide them
- **Field validators recap**: on both our pydantic schemas and our SQLAlchemy table, we use `Field()` to add validation, stuff like:
 - `Field(max_length=100)` -> caps how long a string can be
 - `Field(gt=1, le=25)` -> number has to be greater than 1 and less than or equal to 25
 - `Field(default=SomeEnum.value)` -> a default value if nothing is given
 - `Field(default_factory=datetime.now)` -> runs a function to get the default value fresh every time, instead of computing it once when the class is defined (using `default=datetime.now()` would freeze the time at import time, which is a bug, `default_factory` avoids that)
- **Scalar docs**: fastapi already gives us `/docs` for free (the swagger ui), but theres also a nicer looking alternative called scalar. you just uv add scalar-fastapi, then add a route like this and now you got a prettier docs page too:

```
[ ]: from scalar_fastapi import get_scalar_api_reference
```

```
# route ordering matters, define static routes before dynamic routes
@app.get("/scalar", include_in_schema=False)
async def scalar_html():
    return get_scalar_api_reference(
        openapi_url=app.openapi_url, title="JJ API Documentation"
    )
```